به نام خدا

شبکه‌های کامپیوتری

نیمسال اول سال تحصیلی ۱۴۰۵ – ۱۴۰۶

تمرین کامپیوتری ۱

برنامه‌نویسی سوکت: UT-Chat

UDP Pinger، سرور گفت‌وگوی TCP و سرور FTP

استاد درس: دکتر ناصر یزدانی

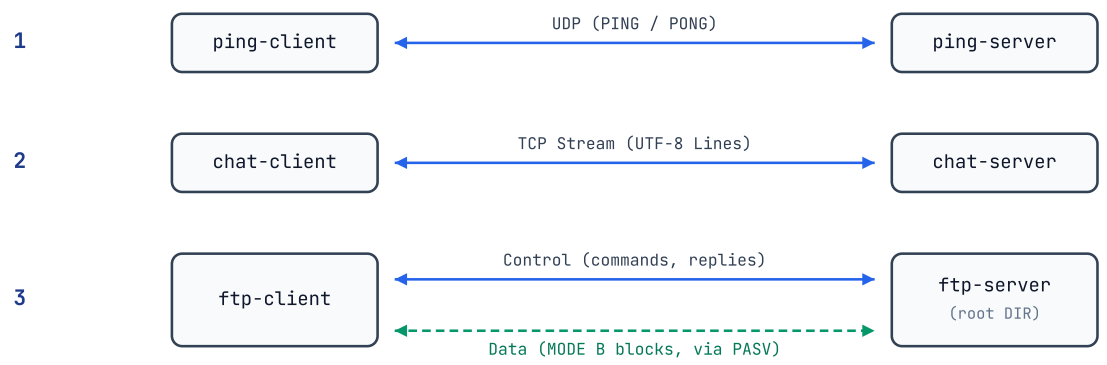
طراحان: صادق عابدی ، محمدمهدی اقلیمی

مهلت ارسال: ۱۴۰۵/۰۸/۱۵، ساعت ۲۳:۵۹

مقدمه

برنامه‌های شبکه‌ای در نهایت روی یک رابط ساده ساخته می‌شوند: **سوکت** . در این تمرین با سوکت‌های UDP و TCP در زبان C/C++ و بدون هیچ کتابخانه آماده‌ای بالاتر از سوکت، در سه بخش برنامه‌های **سرور و کلاینت** را می‌سازید:

بخش	برنامه‌ها (سرور و کلاینت)	موضوع	نمره
۱	`ping-server` `ping-client`	UDP، مهلت زمانی، اتلاف بسته، اندازه‌گیری RTT	۱۵
۲	`chat-server` `chat-client`	TCP، سرور همروند و کلاینت گفت‌وگو با اتاق و پیام خصوصی، مرزبندی پیام روی جریان بایتی	۳۵
۳	`ftp-server` `ftp-client`	FTP، اتصال کنترل و داده، انتقال فایل در بلوک‌های داده (MODE B)	۱۵



شکل ۱: دیاگرام ارتباط کلاینت و سرور در سه بخش تمرین. در بخش ۳ علاوه بر اتصال کنترل، اتصال داده موقت ساخته می‌شود.

نحوه ارزیابی و تحویل: در این تمرین شما باید برای هر سه بخش، هم سمت **سرور** و هم سمت **کلاینت** را مطابق پروتکل‌ها و دستورات مشخص‌شده پیاده‌سازی کنید. صحت عملکرد برنامه‌ها در جلسه **ارائه حضوری** توسط دستیاران آموزشی با اجرای سناریوهای آزمایشی روی کلاینت و سرور شما (و همچنین ابزارهایی مانند `nc`) بررسی و نمره‌دهی می‌شود. بنابراین قالب هر پیام و پاسخ باید دقیقاً مطابق این سند باشد. هر جا ابهامی دیدید، سؤالات خود را در **تلگرام و یا از طریق ایمیل** از دستیاران آموزشی بپرسید.

رابط کامپایل و اجرای برنامه‌ها

پروژه شما باید دارای یک `Makefile` یا اسکریپت `build.sh` برای کامپایل خودکار تمامی بخش‌ها باشد. همچنین می‌توانید یک اسکریپت `run.sh` قرار دهید یا فایل‌های اجرایی را به گونه‌ای بسازید که با آرگومان‌های خط فرمان زیر قابل اجرا باشند:

بخش	فرمان اجرا و آرگومان‌ها
۱	`ping-server --port P --loss L [--seed S]` `ping-client --host H --port P --count N --timeout MS --interval MS`
۲	`chat-server --port P` `chat-client --host H --port P`
۳	`ftp-server --port P --root DIR` `ftp-client --host H --port P`

- پورت‌ها در زمان اجرا به عنوان آرگومان داده می‌شوند؛ هیچ پورتی را در کد ثابت (hardcode) نکنید.
- سرورها در پیش‌زمینه اجرا می‌شوند و روی `0.0.0.0` (یا دست‌کم `127.0.0.1`) گوش می‌دهند.

بخش ۱: UDP Pinger

۱۵ نمره

یک سرور و یک کلاینت ping روی UDP . سرور مثل یک شبکه واقعی بخشی از بسته‌ها را گم می‌کند و کلاینت باید با مهلت زمانی، RTT و اتلاف را اندازه‌گیری کند . هر پیام یک دیتاگرام ASCII با حداکثر ۵۱۲ بایت است؛ `\n` پایانی اختیاری است و فیلدها با دقیقاً یک فاصله جدا می‌شوند.

پیام‌ها

جهت	پیام	توضیح
`Client → Server`	`PING <seq> <token>`	`seq` : عدد صحیح نامنفی ده‌دهی (حداکثر ۱۰ رقم). `token` : ۱ تا ۶۴ نویسه ASCII چاپ‌پذیر (کد ۳۳ تا ۱۲۶) به انتخاب کلاینت.
`Server → Client`	`PONG <seq> <token>`	همان `seq` و `token` درخواست، عیناً.

سرور (`ping-server`)

```
./ping-server --port P --loss L [--seed S]
```

- `--port P` : شماره پورت UDP که سرور روی آن گوش می‌دهد (مثلاً `9000`).
- `--loss L` (پیش‌فرض `0.0`): احتمال اتلاف بسته (عددی بین `0.0` تا `1.0`). هر پیام `PING` معتبر با احتمال `L` نادیده گرفته می‌شود و بی‌پاسخ می‌ماند، وگرنه پاسخ `PONG` فرستاده می‌شود.
- `--seed S` (اختیاری): مقدار اولیه مولد اعداد تصادفی (مانند `srand(S)`) جهت تکرارپذیر کردن سناریوهای تست و ارزیابی. در صورت عدم تعیین، از زمان جاری سیستم استفاده شود.
- سرور با یک سوکت به هر تعداد کلاینت هم‌زمان پاسخ می‌دهد.

کلاینت (`ping-client`)

```
./ping-client --host H --port P [--count N] [--timeout MS] [--interval MS]
```

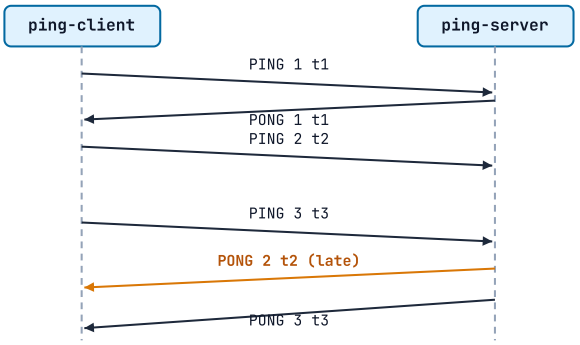
- `--host H` (پیش‌فرض `127.0.0.1`): نشانی IP سرور مقصد.
- `--port P` : پورت UDP سرور مقصد.
- `--count N` (پیش‌فرض `10`): تعداد بسته‌های ارسالی (از `seq=1` تا `N`).
- `--timeout MS` (پیش‌فرض `1000`): مهلت زمانی انتظار برای دریافت پاسخ هر بسته بر حسب میلی‌ثانیه.
- `--interval MS` (پیش‌فرض `0`): زمان استراحت بین ارسال دو پینگ متوالی بر حسب میلی‌ثانیه.

مراحل عملکرد کلاینت برای هر شماره پینگ (از ۱ تا N):

1. **ارسال بسته:** یک دیتاگرام با قالب `PING <seq> <token>` به سرور بفرستید و زمان دقیق ارسال را با ساعت یکنوا (Monotonic Clock) مثل `std::chrono::steady_clock`) ثبت کنید.
2. **انتظار برای پاسخ (تا سقف Timeout):** کلاینت تا `timeout` میلی‌ثانیه منتظر دریافت پاسخ می‌ماند. اگر پاسخی رسید ولی شماره `seq` آن با بسته جاری برابر نبود (مثلاً پاسخ دیرهنگام بسته قبلی بود)، کلاینت آن را نادیده گرفته و دور می‌ریزد و همچنان تا پایان مهلت به انتظار ادامه می‌دهد (شکل ۲).
3. **چاپ نتیجه پینگ:**
   - اگر `PONG` معتبر در مهلت مقرر دریافت شد: مقدار RTT را با کسر زمان ارسال از زمان دریافت حساب کرده و چاپ کنید: `PONG seq=<seq> rtt=<ms> ms` (دقت RTT حداقل سه رقم اعشار باشد).
   - اگر مهلت تمام شد و پاسخی نرسید: عبارت `TIMEOUT seq=<seq>` را چاپ کنید.
4. **وقفه بین پینگ‌ها:** پیش از فرستادن پینگ بعدی، به میزان `interval` میلی‌ثانیه صبر کنید (پس از آخرین پینگ یعنی بسته N-ام نیازی به صبر کردن نیست).

آمار نهایی: در پایان کار، خلاصه نتایج به شکل استاندارد (تعداد ارسالی‌ها، دریافتی‌ها، درصد اتلاف `loss` با یک رقم اعشار و مقادیر min/avg/max زمان RTT) در خروجی چاپ شده و برنامه با کد `0` خاتمه می‌یابد. اگر هیچ پاسخی نرسیده باشد، سطر آخر به صورت `rtt min/avg/max = -/-/- ms` چاپ می‌شود.

```
$ ./ping-client --host 127.0.0.1 --port 9000 --count 5 --timeout 1000
PONG seq=1 rtt=0.412 ms
TIMEOUT seq=2
PONG seq=3 rtt=0.388 ms
PONG seq=4 rtt=0.379 ms
PONG seq=5 rtt=0.401 ms
--- ping statistics ---
sent=5 received=4 loss=20.0%
rtt min/avg/max = 0.379/0.395/0.412 ms
```



شکل ۲: سناریوی پاسخ دیررسیده (Late Reply)

- **تأخیر در شبکه:** پاسخ سرور با تأخیر در مسیر مانده و پس از ارسال `PING 3` به کلاینت می‌رسد.
- **وقوع Timeout:** پاسخ `PING 2` در مهلت مقرر نرسیده و کلاینت خط `TIMEOUT seq=2` را چاپ می‌کند.
- **رفتار کلاینت (Ignore):** چون شماره دریافت شده با بسته جاری همخوانی ندارد (`seq 2 ≠ 3`)، بسته نادیده گرفته شده و کلاینت تا رسیدن پاسخ شماره ۳ منتظر می‌ماند.

خطاها

سمت	پیام / خروجی مورد انتظار (Exact Output)	توضیح رویداد
`Server`	`ERR BAD_REQUEST`	دیتاگرام نامعتبر (تعداد فیلد غلط، دستوری غیر از `PING` ، خارج از قالب یا دارای نویسه‌های غیر ASCII)؛ همیشه و بدون اعمال loss فرستاده می‌شود.
`Client`	`TIMEOUT seq=<seq>`	پایان مهلت زمانی بدون دریافت پاسخ از سرور؛ کلاینت این پیام را در stdout چاپ کرده و به سراغ شماره پینگ بعدی می‌رود.
`Client`	`Ignore`	دریافت پاسخ دیرهنگام (مربوط به seq قبلی)، بسته تکراری یا پیام ناشناخته؛ بسته نادیده گرفته شده و کلاینت تا اتمام مهلت جاری منتظر می‌ماند.

بخش ۲: سرور و کلاینت گفت‌وگوی TCP

۳۵ نمره

سرور (`chat-server`)

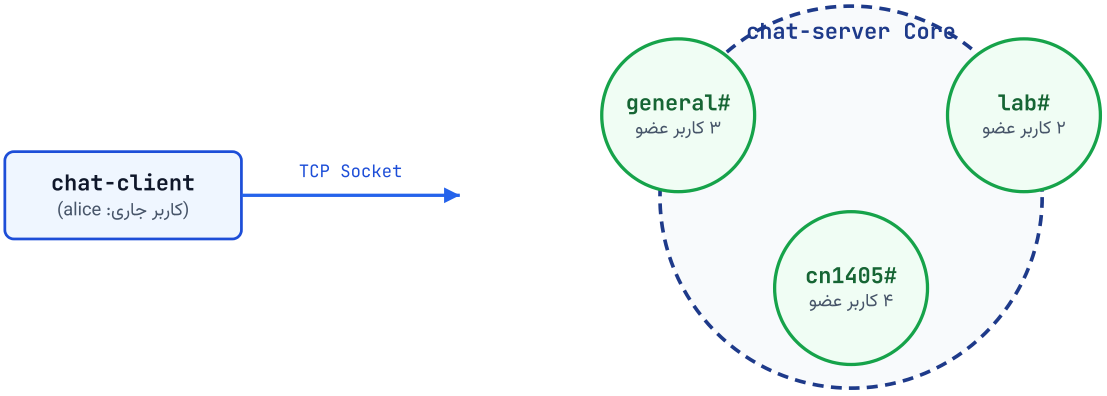
```
./chat-server --port P
```

- `--port P` : پورت TCP که سرور روی آن به درخواست‌های اتصال ورودی گوش می‌دهد (مثلاً `8080`).
- سرور به صورت همروند اتصال‌های TCP را پذیرفته، کاربران را بر اساس نام مستعار مدیریت کرده و پیام‌های هر کاربر را در اتاق‌های مربوطه توزیع می‌کند (با الهام از پروتکل استاندارد IRC).

کلاینت (`chat-client`)

```
./chat-client --host H --port P
```

- `--host H` (پیش‌فرض `127.0.0.1`): نشانی IP سرور چت.
- `--port P` : شماره پورت TCP سرور چت.
- برنامه کلاینت باید به سرور متصل شود، دستورات کاربر را از `stdin` خوانده و به سرور بفرستد، و هم‌زمان پیام‌ها و رویدادهای ناهمگام دریافتی از سرور را بدون بلاک شدن در `stdout` چاپ کند (با استفاده از `poll` / `select` یا دو ترد مجزا). سرور علاوه بر کلاینت شما باید با دستور `nc 127.0.0.1 P` نیز سازگار باشد.



شکل ۳: معماری سرور و اتاق‌های گفت‌وگو. هر کلاینت با اتصال به سرور می‌تواند در یک یا چند اتاق به صورت هم‌زمان عضو شود.

قالب خطوط و پروتکل ارتباطی

- **پایان‌دهنده خطوط (Framing):** تمام پیام‌ها در هر دو جهت باید متنی، با کدگذاری `UTF-8` و ختم‌شده به کاراکتر `\\n` (کد ASCII شماره ۱۰) باشند. سرور باید `\\r\\n` را نیز به درستی بپذیرد (کاراکتر `\\r` جزو داده نیست). ارسال خط خالی بی‌اثر بوده و نادیده گرفته می‌شود.
- **سقف طول پیام:** حداکثر طول هر خط ۱۰۲۴ بایت (بدون احتساب `\\n`) است. اگر خطی طولانی‌تر فرستاده شود، سرور خطای `ERR 413 LINE_TOO_LONG` داده، باقیمانده بایت‌های آن خط را دور می‌ریزد و اتصال را باز نگه می‌دارد.
- **حساسیت و فاصله‌ها:** نام دستورات باید با حروف بزرگ (Case-sensitive) باشد و بین دستور و آرگومان‌ها دقیقاً یک فاصله قرار گیرد. در فیلدهای متنی مثل `<text>` فاصله‌های داخل متن حفظ شده و متن نباید خالی باشد.

- **نام‌گذاری مجاز:** نام مستعار (Nick) باید ۱ تا ۱۶ نویسه از `[A-Za-z0-9_]` باشد. نام اتاق‌ها با کاراکتر `#` شروع شده و شامل ۱ تا ۳۱ نویسه از `[a-z0-9_-]` است (مانند `#cn1405`).

دستورات (Client → Server)

دستور	توضیح	پاسخ موفق (فقط به فرستنده)
`NICK <nick>`	ثبت نام مستعار (فقط یک بار و پیش از هر دستور دیگر)	`OK NICK <nick>`
`JOIN <room>`	پیوستن؛ اتاق در اولین `JOIN` ساخته می‌شود و هیچ‌گاه حذف نمی‌شود	`OK JOIN <room>`
`PART <room>`	ترک اتاق	`OK PART <room>`
`MSG <room> <text>`	پیام به بقیه اعضای اتاق (نه به خود فرستنده)	`OK MSG <room> <id>`
`PRIVMSG <nick> <text>`	پیام خصوصی	`OK PRIVMSG <nick>`
`LIST`	همه اتاق‌ها (حتی خالی) به ترتیب نام	`OK LIST <n>` `+ n × ROOM <room> <count>`
`WHO <room>`	اعضای اتاق به ترتیب نام (عضویت لازم نیست)	`OK WHO <room> <n>` `+ n × USER <nick>`
`PING`	بررسی برقراری اتصال	`PONG`
`QUIT`	سپس سرور اتصال را می‌بندد	`OK BYE`

تا پیش از ثبت نام مستعار، فقط `NICK` ، `PING` و `QUIT` پذیرفته می‌شوند. ترتیب بررسی در خطای یک خط: `LINE_TOO_LONG` ← `UNKNOWN_COMMAND` ← `NOT_REGISTERED` ← `BAD_SYNTAX` ← سپس خطاهای معنایی (`403` ، `404` ، `409`).

پاسخ ها و رویداد ها (Server → Client)

علاوه بر پاسخ‌های جدول بالا، سرور این خطوط ناهمگام را هر زمان ممکن است بفرستد:

خط	معنا
`HELLO UT-CHAT/1.0`	بلافاصله پس از برقراری اتصال.
`FROM <room> <nick> <id> <text>`	پیام شماره `<id>` اتاق از طرف `<nick>` .
`PRIV <nick> <text>`	پیام خصوصی از طرف `<nick>` .
`EVENT JOIN <room> <nick>`	کاربری به اتاق شما پیوست (به خود او فرستاده نمی‌شود).
`EVENT LEAVE <room> <nick>`	کاربری از اتاق شما رفت (`PART` ، `QUIT` یا قطع اتصال).

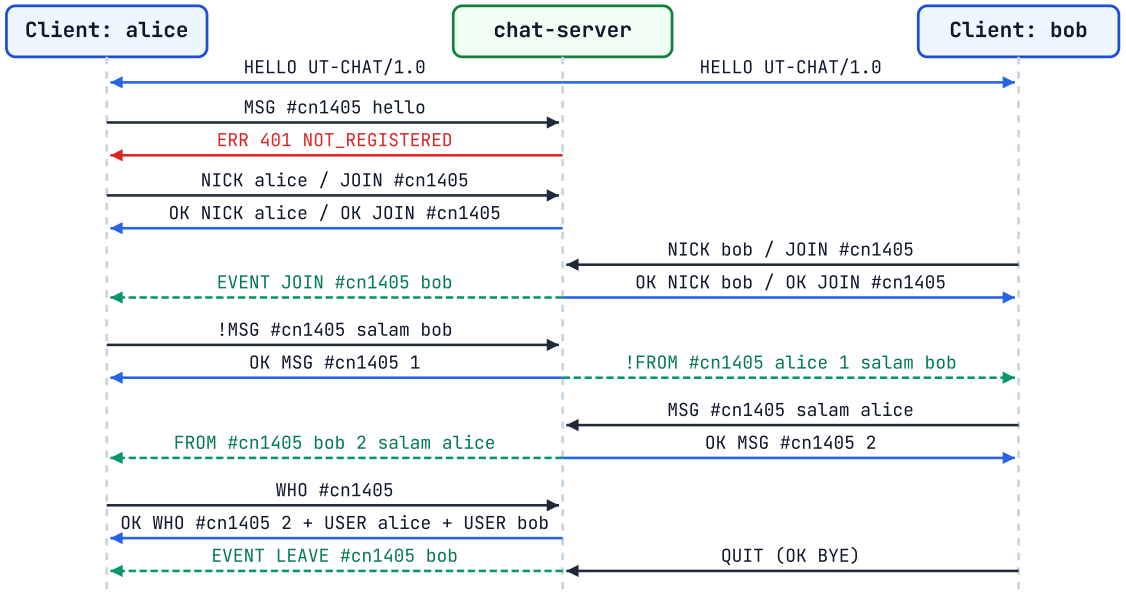
تضمین‌ها: هر اتاق شمارنده خودش را دارد (اولین پیام `1` ، بدون جاافتادگی)؛ همه اعضای یک اتاق پیام‌ها را به ترتیب صعودی شناسه می‌بینند حتی وقتی چند نفر هم‌زمان می‌فرستند. پاسخ دستورهای هر کلاینت به ترتیب دستورها می‌آید و خطوط ناهمگام هرگز وسط یک پاسخ چندخطی (`LIST` ، `WHO`) قرار نمی‌گیرند.

خطاها

هر خطا یک خط `ERR <code> <TOKEN>` است. سمت سرور این خطوط را می‌فرستد؛ سمت کلاینت کد و token را دریافت و نمایش می‌دهد و اتصال پس از خطا باز می‌ماند.

کد	token	توضیح
400	UNKNOWN_COMMAND	نام دستور ناشناخته است
400	BAD_SYNTAX	تعداد/قالب آرگومان‌ها نادرست (مثلاً اتاق بدون `#` ، متن خالی)
401	NOT_REGISTERED	دستوری غیر از `NICK` / `PING` / `QUIT` پیش از ثبت نام
403	NOT_IN_ROOM	عملیات نیازمند عضویت در اتاق است (`MSG` ، `PART`)
404	NO_SUCH_NICK / NO_SUCH_ROOM	کاربر متصل یا اتاقی با این نام نیست (`PRIVMSG` ، `WHO`)
409	NICK_IN_USE / ALREADY_REGISTERED / ALREADY_IN_ROOM	نام را دیگری دارد / دوباره `NICK` / دوباره `JOIN`
413	LINE_TOO_LONG	خط بیش از ۱۰۲۴ بایت

مثال و دیاگرام تبادل پیام در سرور گفت‌وگو



شکل ۳: دیاگرام تعامل هم‌زمان دو کلاینت (`alice` و `bob`) با `chat-server` شامل ثبت‌نام، پیوستن به اتاق، شماره‌گذاری پیام‌ها و رویدادهای ناهمگام.

```
# --- alice از دید کلاینت ---+
S: HELLO UT-CHAT/1.0
C: MSG #cn1405 hello
S: ERR 401 NOT_REGISTERED
C: NICK alice
S: OK NICK alice
C: JOIN #cn1405
S: OK JOIN #cn1405
S: EVENT JOIN #cn1405 bob               # bob joins #cn1405 from another connection
C: MSG #cn1405 salam bob!
S: OK MSG #cn1405 1                     # message id=1 confirmed to sender
S: FROM #cn1405 bob 2 salam alice       # bob's reply arrives with id=2
C: WHO #cn1405
S: OK WHO #cn1405 2
S: USER alice
S: USER bob
C: MSG #lab anyone?
S: ERR 403 NOT_IN_ROOM
S: EVENT LEAVE #cn1405 bob              # bob sends QUIT or disconnects
C: QUIT
S: OK BYE
```

الزامات همروندی و استحکام

1. **همروندی:** سرور هم‌زمان به چندین کلاینت متصل خدمت می‌دهد. دو مدل رایج: thread به ازای هر کلاینت با قفل روی داده مشترک، یا حلقه رویداد با `select` / `poll` / `epoll` .
2. **جریان بایتی:** TCP مرز پیام نمی‌شناسد . یک `recv` ممکن است نصف خط، چند خط، یا حتی نیمی از یک نویسه چندبایتی UTF-8 (حروف فارسی) را برگرداند. بایت‌ها را در بافر جمع کنید و فقط خطوط کامل را پردازش کنید.
3. **قطع ناگهانی:** کلاینت ممکن است بدون `QUIT` و در هر لحظه (`FIN` یا `RST`) قطع شود. این مثل `QUIT` رفتار می‌شود (از جمله `EVENT LEAVE` و آزاد شدن نام) و سرور هرگز از کار نمی‌افتد (مثلاً با `SIGPIPE`).
4. **ترتیب:** شناسه دادن و قرار دادن پیام در صف ارسال همه اعضا باید یک کار اتمی باشد.

بخش ۳: سرور و کلاینت FTP

۱۵ نمره

سرور (`ftp-server`)

```
./ftp-server --port P --root DIR
```

- `--port P` : پورت شنونده TCP برای برقراری اتصال کنترل (Control Connection).
- `--root DIR` : مسیر پوشه محلی ذخیره‌سازی فایل‌ها. ساختار پوشه کاملاً تخت (Flat) است و زیرپوشه نداریم. تمام فایل‌ها مستقیماً با نام مجاز (`[A-Za-z0-9._-]{1,64}`) درون این پوشه قرار می‌گیرند.
- دستورات `LIST` و `RETR` همواره وضعیت واقعی دیسک را در لحظه نشان می‌دهند (عدم نگهداری لیست جداگانه در حافظه).
- دستور `STOR` اتمیک (Atomic) است؛ فایل ناقص یا آسیب‌دیده ذخیره نمی‌شود و تا پیش از دریافت کامل بلوک EOF بر روی دیسک قرار نمی‌گیرد.

کلاینت (`ftp-client`)

```
./ftp-client --host H --port P
```

- `--host H` (پیش‌فرض `127.0.0.1`): نشانی IP سرور FTP.
- `--port P` : پورت اتصال کنترل سرور FTP.
- کلاینت اتصال کنترل را با سرور برقرار کرده و دستورات کاربر را از `stdin` دریافت می‌کند. برای دستورات نیازمند انتقال داده (`LIST` ، `RETR <file>` و `STOR <file>`)، کلاینت ابتدا پورت موقت داده را با ارسال `PASV` از سرور دریافت کرده، اتصال داده جدیدی می‌سازد و بسته‌های فایل را در قالب بلوک‌های `MODE B` منتقل می‌کند.

FTP در یک نگاه (معماری دو-اتصاله و خارج از باند)

پروتکل FTP از طراحی **کنترل خارج از باند (Out-of-band Control)** بهره می‌برد و برخلاف HTTP، دارای دو اتصال TCP مجزا است: **۱) اتصال کنترل:** در تمام طول نشست باز می‌ماند و تنها برای ارسال خطوط متنی دستورات و پاسخ‌ها استفاده می‌شود. **۲) اتصال داده:** برای هر بار جابه‌جایی فایل به شکل موقت ساخته شده و در پایان انتقال بسته می‌شود. در حالت **Passive**، کلاینت با ارسال دستور `PASV` از سرور یک پورت موقت می‌گیرد و به آن متصل می‌شود. در حالت **MODE B**، داده‌ها داخل بلوک‌هایی با سرآیند ۳ بایتی (توصیف‌گر ۱ بایت + شمارنده طول ۲ بایت) ارسال شده و پایان انتقال با پرچم EOF مشخص می‌گردد.

دستورات (Client → Server)

خطوط به `\\r\\n` ختم می‌شوند (`\\n` تنها هم پذیرفته شود)؛ نام دستور به بزرگی/کوچکی حساس نیست. آرگومان هر دستور بقیه خط پس از اولین فاصله است. از آنجا که نام فایل‌ها نباید شامل فاصله باشد، ارسال نامی با فاصله مانند `STOR bad name.txt` نامعتبر بوده و خطای `553` دارد؛ کد `501` صرفاً برای عدم ارسال آرگومان است.

دستور	توضیح عملکرد	پاسخ و خروجی موفق
`USER <name>`	ارسال نام کاربری (هر نامی پذیرفته می‌شود).	`331 Password required`
`PASS <password>`	ارسال رمز عبور پس از `USER` (هر رمزی پذیرفته می‌شود).	`230 Logged in`
`MODE B`	تنظیم حالت انتقال بر روی بلوک (تنها حالت مجاز).	`200 OK`
`PASV`	درخواست پورت موقت؛ پاسخ حاوی `h1.h2.h3.h4` و پورت `p1×256+p2` است. هر انتقال نیازمند یک `PASV` تازه است.	`227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)`

`LIST`	دریافت فهرست فایل‌های موجود در پوشه ریشه سرور.	`<filename_1> <size_1>`
		`<filename_2> <size_2>`
		`...`

`RETR <file>`	دانلود فایل مشخص‌شده از سرور.	`150 Opening data connection` `226 Transfer complete`
`STOR <file>`	آپلود فایل بر روی سرور (جایگزینی در صورت وجود فایل).	`150 Opening data connection` `226 Transfer complete`
`QUIT`	خروج از سامانه؛ سرور اتصال کنترل را می‌بندد.	`221 Goodbye`

بلافاصله پس از برقراری اتصال کنترل، سرور پیام `220 UT-FTP ready` را می‌فرستد. کلاینت وضعیت را بر اساس کد عددی پاسخ ارزیابی می‌کند.

خطاها (پاسخ‌های سرور)

کد	توکن خطا (Token)	توضیح رویداد
500	UNKNOWN_COMMAND	دستور ارسال‌شده ناشناخته است (پیش از بررسی لاگین ارزیابی می‌شود).
501	SYNTAX_ERROR_PARAMS	آرگومان لازم داده نشده است (مانند ارسال USER ، RETR ، STOR یا MODE بدون آرگومان).
503	BAD_SEQUENCE	ترتیب اجرای دستورات نادرست است: ارسال PASS پیش از USER ، یا انتقال بدون اجرای MODE B .
504	COMMAND_NOT_IMPLEMENTED	حالت MODE تعیین‌شده مقداری غیر از B باشد.
530	NOT_LOGGED_IN	کاربر لاگین نکرده و دستوری غیر از USER ، PASS یا QUIT را فرستاده است.
553	NAME_NOT_ALLOWED	نام فایل در دستورات STOR یا RETR نامعتبر است (مانند نام دارای فاصله، کاراکترهای غیرمجاز مانند ../ یا طول بیش از ۶۴ نویسه).
425	CANT_OPEN_DATA_CONNECTION	دستور انتقال بدون اجرای PASV صادر شده یا پورت داده قبلی مصرف شده است.
550	FILE_NOT_FOUND	فایل درخواستی برای دانلود (دستور RETR) در پوشه ریشه سرور وجود ندارد.
426	CONNECTION_CLOSED	انتقال قطع شد: دریافت بلوک نامعتبر یا بسته شدن زودهنگام اتصال داده پیش از دریافت بلوک EOF (پس از ارسال پاسخ 150).

ترتیب بررسی خطاها برای دستورات انتقال: 530 ← 501 ← 553 ← 503 ← 425 و سپس 550 است. خطاها اتصال کنترل را نمی‌بندند. در صورت دریافت هر پاسخ 4xx یا 5xx ، دستور ناموفق بوده و کلاینت نباید هیچ داده‌ای روی سوکت داده ارسال کند.

اتصال داده و بلوک‌ها (MODE B)

هر بلوک: سرآیند سه‌بایتی (توصیف‌گر ۱ بایت + شمارنده طول ۲ بایت big-endian) و سپس دقیقاً همان تعداد بایت داده:



شکل ۴: قالب بلوک MODE B. هر بلوک یک بسته است؛ آخرین بلوک پرچم EOF دارد.

- **هر انتقال:** کلاینت `PASV` می‌فرستد، به نشانی `227` وصل می‌شود (پیش یا پس از فرستادن `LIST` / `RETR` / `STOR` ؛ سرور هر دو ترتیب را تحمل کند) و دستور را می‌فرستد؛ سرور `150` می‌فرستد، داده جابه‌جا می‌شود و در پایان `226` می‌آید.
- **فرستنده (سرور در `RETR` و `LIST` ؛ کلاینت در `STOR`):** داده در بلوک‌های ۱۰۲۴ بایتی ارسال می‌شود؛ هر بلوک غیر از آخری دقیقاً ۱۰۲۴ بایت دارد و آخرین بلوک پرچم EOF (`0x40`) و ۱ تا ۱۰۲۴ بایت باقی‌مانده را حمل می‌کند (فایل خالی: یک بلوک `40 00 00` است). پس از بلوک EOF فرستنده اتصال داده را می‌بندد.
- **گیرنده (سرور در `STOR` ؛ کلاینت در `RETR` و `LIST`):** بلوک‌هایی با هر اندازه `0` تا `65535` را (از جمله یک بلوک EOF خالی پس از داده) تا بلوک EOF می‌پذیرد.
- **خطا:** توصیف‌گر غیر از `0x40` / `0x00` یا بسته شدن اتصال داده پیش از EOF ← کد `426` . دستور `STOR` اتمی است: فایل تنها پس از بلوک EOF دیده می‌شود؛ در خطا فایلی ساخته نمی‌شود و فایل قبلی با همان نام دست‌نخورده می‌ماند.
- چند کلاینت می‌توانند هم‌زمان و با اتصال داده‌های جدا انتقال انجام دهند.

```
S: 220 UT-FTP ready
C: USER alice               S: 331 Password required
C: PASS secret              S: 230 Logged in
C: MODE B                   S: 200 OK
C: PASV                     S: 227 Entering Passive Mode (127,0,0,1,195,80)
                            # client connects to 127.0.0.1:50000 (195*256+80)
C: STOR notes.txt           S: 150 Opening data connection
data: 00 0400 <1024 B>  00 0400 <1024 B>  40 01C4 <452 B>  (client closes)
                            S: 226 Transfer complete
C: RETR missing.txt         (after PASV) S: 550 FOUND_NOT_FILE
C: QUIT                     S: 221 Goodbye
```

ارزیابی و ارائه حضوری

بخش	موارد مورد ارزیابی	نمره
۱: UDP Pinger	پیاده‌سازی کامل `ping-server` و `ping-client`	۱۵
۲: گفت‌وگوی TCP	پیاده‌سازی کامل `chat-server` و `chat-client` همروندی .	۳۵
۳: انتقال فایل FTP	پیاده‌سازی کامل `ftp-server` و `ftp-client` با `MODE B`	۱۵
جمع پیاده‌سازی و تست‌ها	—	**۶۵**
ارائه حضوری و مرور کد (فردی)	تسلط بر کد، معماری و پاسخ به پرسش‌های مفهومی	۳۵

ارائه حضوری

هر گروه در یک جلسه حدود **۳۰ دقیقه‌ای**حاضر می‌شود . حضور هر دو عضو الزامی و نمره فردی است. در جلسه تحویل، سناریوهای تست از پیش آماده‌شده روی فایل ارسالی شما اجرا می شود.

قوانین و ارسال

- **گروه دونفره:** هر دو عضو باید در همه بخش‌ها مشارکت کنند و در ارائه به کل کد مسلط باشند.
- **مخزن Git:** یک مخزن خصوصی بسازید و نام کاربری دستیاران آموزشی در GitHub/GitLab را به آن اضافه کنید؛ تاریخچه commitها از هر دو عضو دیده می‌شود.
- **مهلت ارسال** مهلت ارسال پروژه تا ۱۴۰۵/۰۸/۱۵ می باشد . به ازای هر روز تاخیر 5 درصد از نمره ی شما کم می شود . شما میتوانید حداکثر تا 5 روز با تاخیر پروژه را تحویل بدهید .
- **زبان:** صرفاً **C/C++** (استاندارد و قابل کامپایل و اجرا با `GCC/G++` در محیط لینوکس / WSL).
- **مجاز:** سوکت‌های کتابخانه استاندارد زبان، threadها، `select` / `poll` / `epoll` و ورودی/خروجی ناهمگام در سطح سوکت، و کتابخانه‌های عمومی مثل ساختمان داده‌ها.
- **ممنوع:** هر کتابخانه یا چارچوبی که بخشی از پروتکل را برای شما انجام دهد (`ftplib`، Apache Commons Net، ZeroMQ، WebSocket، gRPC، `pyftpdlib` و مشابه آن‌ها)؛ همچنین اجرای برنامه‌های خارجی مثل `nc` از درون برنامه.
- استفاده از دستیارهای هوش مصنوعی برای یادگیری، پرسیدن مفاهیم و رفع اشکال مجاز است؛ اما مسئولیت تک‌تک خطوط کد با شماست و در ارائه باید بتوانید هر بخش از کد را توضیح دهید. در `README` مخزن بنویسید از چه ابزارهایی و برای چه کاری استفاده کرده‌اید. کدی که نتوانید توضیح دهید کد شما محسوب نمی‌شود. استفاده از کد دیگران (همین درس، سال‌های قبل، مخازن عمومی) مجاز نیست و با هر دو طرف مطابق قوانین دانشکده برخورد می‌شود. منابع مورد استفاده (کتاب، RFC، راهنمای Beej) را می‌توانید در `README` مخزن بیاورید.
- **ارسال:** یکی از اعضا تا مهلت مقرر یک فایل `CA1_<SID1>_<SID2>.zip` را در سامانه elearn بارگذاری می‌کند. ریشه آن شامل کدهای منبع، فایل `Makefile` (یا `build.sh` / `run.sh`) و `README.md` (شامل توضیح کوتاه معماری و نحوه کامپایل و اجرا) است. فایل اجرایی و خروجی کامپایل را در فایل ارسالی نگذارید (حداکثر ۲۰ مگابایت).